

GAIADA Search-Marketing

SEO, SEM, and GEO/AEO as one platform-nest module vertical and department console — not a fleet of adopted external apps. The professional moat is data, not UI: one provider abstraction, self-hosted crawlers, local-first AI, and a dual-mode execution path for every live-account action.

DOCUMENT Engineering Blueprint v1.1	MODULE KEY search (search-marketing)	DATE 2026-07-23	OWNER SEO Dept · Platform
STATUS ● PLANNED · V0.0.0	CLASSIFICATION Internal		

Contents

00	Executive Summary	08	AI Design
01	Foundation & Cost Model	09	Console UX
02	Scope & Pillars	10	ERP Integration Points
03	System Overview	11	Automation Flows
04	Trust Zones & Network	12	Trust & Security
05	Domain Model & Schema	13	Rollout & Tickets
06	Data-Provider Abstraction	14	Open Questions & Decisions
07	Fork/Adapt Plan per Repo	15	Glossary

Sections are numbered; each begins on a new page. Use the PDF bookmarks or the page footer to navigate.

§00 Executive Summary

What we are building, why it lives inside platform-nest rather than as adopted tools, and the shape of the decisions already locked.

Search-marketing becomes a **platform-nest module vertical** (`ModuleContract` key `search` , tables `search_*`) plus a **department console** on the dept-interface-template — the same shape as Web Dev, not a second UI bolted on top of somebody else's SaaS. The professional moat in this discipline is data, not workflow, so the architecture follows that truth end to end.

1 · PROVIDER ABSTRACTION

`SearchDataProvider` drivers (DataForSEO primary, Semrush premium, free-scraper fallback) behind a Postgres cache and a per-client usage/cost ledger that enforces budget stop-losses *before* money is spent. A human sets which tools run at all, per client.

2 · SELF-HOSTED CRAWLERS

SEONaut, open-seo-crawler, and Unlighthouse run as job-mode workers — \$0 API cost, egress-guarded, ingested into RLS-scoped `search_audits` . Their own UIs and auth are never exposed.

3 · MINE, DON'T RUN

`open-seo` and SerpBear are data-only references: retrofitting single-tenant apps with FORCE-RLS + Keycloak + Cerbos costs more than native implementation, and their UIs are redundant behind our console.

4 · LOCAL-FIRST AI

Hermes via ai-gateway-go handles bulk work (clustering, intent-tagging, triage, first drafts); Claude is reserved for client-facing polish only. GEO/brief RAG rides the existing WS8 knowledge store.

5 · WS4-GATED, DUAL-MODE EXECUTION

Everything touching a live site or live ad spend is approval-gated. Every live action ships as a **manual twin** (export, human applies, zero OAuth, P3) and an **automated twin** (API push behind WS4, P4, committed) — the human picks per action.

6 · GEO/AEO FIRST-CLASS

AI-visibility snapshots, extractability audits, and citation-oriented tooling ride the same entities and provider abstraction as SEO/SEM — enabled per client like every other paid pull, not a bolt-on premium tier.

Build order is cost-honest: **P0 contracts** → **P1 \$0-value (crawls + AI on own data)** → **P2 paid data (needs the \$50 DataForSEO deposit)** → **P3 SEM planning/reports + manual-apply path** → **P4 live-ads OAuth writes (committed) + decision-gated extras**. 26 tickets P0–P3 plus 2 committed P4 tickets, /army-ready (\$13).

READING THIS DOCUMENT

This is a **design blueprint** — nothing in it exists as code yet. It targets a future `search-marketing` module registered as `0.0.0 · PLANNED` in `docs/modules/MODULES.md` on approval. Primary inputs: `seo-sem-design.md` (v1.1, the architecture) and its companion `seo-sem-foundation.md` (research + the locked cost model, folded into §01 below).

§01 Foundation & Cost Model

What SEO and SEM actually are, what clients pay for, the professional toolchain, and the locked provider-cost decision this design conforms to.

1.1 · Two disciplines, converging on a third

Both SEO and SEM are **search marketing** — capturing demand from people (and now AI answer engines) actively searching. They differ in how visibility is won and how fast.

	SEO — ORGANIC	SEM — PAID (SEA)
Mechanism	Earn organic rankings	Buy placement via ad auctions
Speed	Slow — compounding asset over months	Instant — traffic the day a campaign goes live
Cost shape	Labor-heavy, no per-click cost	Media spend + management fee; stops when budget stops
Durability	Durable asset that keeps paying	Rented traffic; ends with the budget

2026 FRAMING

The modern view is not "free vs paid" — it's **portfolio management of search intent** across SERP formats *and* AI answer engines. A third discipline has emerged: **GEO / AEO / LLMO** — optimizing to be *cited by* ChatGPT, Google AI Overviews, Gemini, Claude, Perplexity. This design treats GEO/AEO as first-class, not an afterthought.

1.2 · What clients actually pay for

Clients don't buy rankings or clicks — they buy **business outcomes**. KPIs sit in four SEO buckets (visibility → traffic → engagement → **outcome**) and four SEM metrics (CPC, CTR, Quality Score, **ROAS**). The ERP must model this as an **outcome-tracked engagement**, not a task list — intake the goal, audit current state, execute, measure against a committed KPI, report the outcome.

DISCIPLINE	PRIMARY CLIENT OUTCOME	TIMEFRAME
SEO	Durable, compounding organic traffic → leads/revenue at falling CPA	3–12 months

DISCIPLINE	PRIMARY CLIENT OUTCOME	TIMEFRAME
SEM	Immediate qualified traffic at a controllable, predictable CPA/ROAS	Days
GEO/AEO	Brand presence & citation inside AI answers — the new zero-click surface	Emerging

1.3 • The professional stack — and the moat

A practical stack = Search Console + GA4 + one crawler + one research platform + one reporting layer + Google Ads for SEM. The commercial layer is where the real cost lives:

TOOL	ROLE	NOTE
Semrush	All-in-one SEO+SEM data	Strongest all-rounder; an MCP connector exists but needs OAuth (OQ-3)
Ahrefs	Backlink data reference standard	Best-in-class link index
Screaming Frog	Desktop technical crawler	De-facto site-audit spider
Looker Studio	Reporting / dashboards	Free; pulls GSC/GA4/Semrush/Ahrefs

KEY INSIGHT

The professional moat is **data** (SERP volumes, backlink index), not the UI. Open-source tools replicate the workflow; they still need a paid data API for volume/difficulty/backlink numbers. Our build = OSS workflow + pluggable data provider — never adopted external UIs.

1.4 • Where AI genuinely helps

TASK	AI ROLE	MODEL CLASS
Keyword → intent clustering	Group thousands of keywords into intent/topic clusters	Embeddings + small LLM
Content briefs / topic maps	Generate briefs, outlines, entity coverage from SERP + competitors	Mid/large LLM
Technical audit triage	Summarize crawl findings, prioritize by impact, draft fixes	Mid LLM
SEM search-term / negatives	Classify terms, propose negatives, draft RSA copy	Mid LLM + rules
GEO/AEO	Structure content for RAG extraction & citation; track AI mentions	LLM + retrieval

TASK	AI ROLE	MODEL CLASS
Reporting narrative	Turn metrics into a client-readable story	Mid LLM

Human-in-the-loop is mandatory — AI drafts, human approves, routed through the existing WS4 approvals surface, exactly like schema drafting and WebDev.

1.5 · LOCKED decision — provider & cost model (2026-07-23)

CONTEXT

Gaiada serves many smaller clients with variable needs from Indonesia; IDR/USD pressure means the stack must be cheapest-yet-professional. This decision is **locked** and this design does not relitigate it.

DataForSEO (Standard queue) is primary; Semrush MCP is an optional premium upsell, behind a pluggable provider abstraction. Rationale: pay-as-you-go maps to per-client billing with no dead subscription floor, and it's a data layer suited to build-it-yourself.

Cost-control levers

- 1. AI bulk work → local Hermes; paid Claude reserved for client-facing polish only.
- 2. Default DataForSEO Standard queue (\$0.0006/SERP, ~5-min) — 3× cheaper than Live.
- 3. Self-hosted crawlers do all audit/crawl/speed work — \$0 API cost.
- 4. Cache SERP/volume in Postgres — no re-querying the same keyword within a window.
- 5. Meter per client in the ERP; one shared \$50 DataForSEO deposit pool across all clients.

TIER	#	SCOPE	\$/CLIENT	SUBTOTAL
Light	40	Rank + monthly research (SEO <i>or</i> SEM)	~\$3	\$120
Standard	40	Full SEO <i>or</i> full SEM	~\$8	\$320
Heavy	20	Full SEO + SEM (more kw/competitors/backlinks)	~\$18	\$360
DataForSEO total	100			≈ \$800/mo
AI (local-first + selective cloud)				+\$50–150/mo
Grand total				≈ \$850–950/mo

Scenario range: **floor ~\$600** (cache-heavy, all-Standard, all-local AI) · **central ~\$900** · **ceiling ~\$1,800** (all 100 full-heavy on Live queue). Blended **~\$8–10/client/mo**.

METRIC	USD	IDR @ 22K
Floor (100 clients)	\$600/mo	~Rp 13.2M/mo
Central (100 clients)	\$900/mo	~Rp 19.8M/mo
Ceiling (100 clients)	\$1,800/mo	~Rp 39.6M/mo
Blended per client	\$8–10/mo	~Rp 176k–220k/mo

Even buffered at Rp 22,000/USD, data cost is a rounding error against a professional retainer. **vs. Semrush for the same 100 clients:** \$500/mo base + heavy unit add-ons for programmatic use → ~\$2,000–5,000+/mo, and not meterable per client. DataForSEO is ~3–5× cheaper and bills the way we sell.

SIBLING DELIVERABLE

This design follows the same rigor as the [WebDesk Engineering Blueprint](#) — where WebDesk is the *website execution* zone, search-marketing is a *data + judgment* subsystem that lives almost entirely inside the platform (Zone A).

§02 Scope & Pillars

All three pillars first-class from v1 — what ships, what's deliberately deferred, and how this fits prior platform decisions.

PILLAR	V1 DELIVERS	DEFERRED
SEO	Technical audits (crawl/CWV/links), keyword research + intent clustering, rank tracking, backlink snapshots, content briefs + on-page drafts, outcome-tracked engagements + KPI targets, client reports	Log-file analysis; local SEO (GBP/citations); Bing Webmaster; disavow workflows (human-only); full backlink inventory (snapshots only)
SEM	Campaign/ad-group/ad <i>planning objects</i> from keyword clusters; AI RSA drafts; negative-keyword proposals; imported performance; budget-pacing alerts; dual-mode execution for every live action	Microsoft Ads; offline conversion import; automated bid strategies
GEO / AEO	AI-visibility snapshots (brand cited across ChatGPT, AI Overviews, Gemini, Claude, Perplexity), extractability audit kind, citation-oriented brief guidance (RAG over crawled content). Enabled per client via the tool-scope config	Ansvisor-style dedicated tracker fork; automated GEO content rewriting at scale

Non-goals (v1)

- **No adopted external UIs** — the console is the only operator surface.
- **No direct vendor AI calls** — everything through ai-gateway-go (locked).
- **No client-facing portal changes** — reports deliver as files (Shared Drive + [deliverables](#)).

- **No Matomo** — Umami chosen (MIT, lighter), and even Umami is decision-gated (OQ-5).
- **No payroll/billing changes** — provider costs are metered; invoicing is the existing billing module's concern.

Fit with prior decisions

Conforms to the **ERP holding-OS vision** (shared-service department serving N companies): the module uses the standard enablement OR-clause (`enabled_modules` OR active `service_assignment`) and the WSD-3 module-sliced RLS wall.

D-10 · CONSOLE RATIFIED 2026-07-23

Supersedes one line of the dept-console IA plan (which sketched a single "Optimize" tab). The department keeps the name **SEO**; its console follows the dept-interface-template exactly as Web Dev does — universal spine plus **three craft groups** (Accounts / Optimize / Campaigns), since SEM cannot honestly fit in four SEO sub-tabs.

§03 System Overview

All state and judgment live in Zone A. Three arrows cross to the internet: crawler egress, provider API calls, and notification-only inbound webhooks.

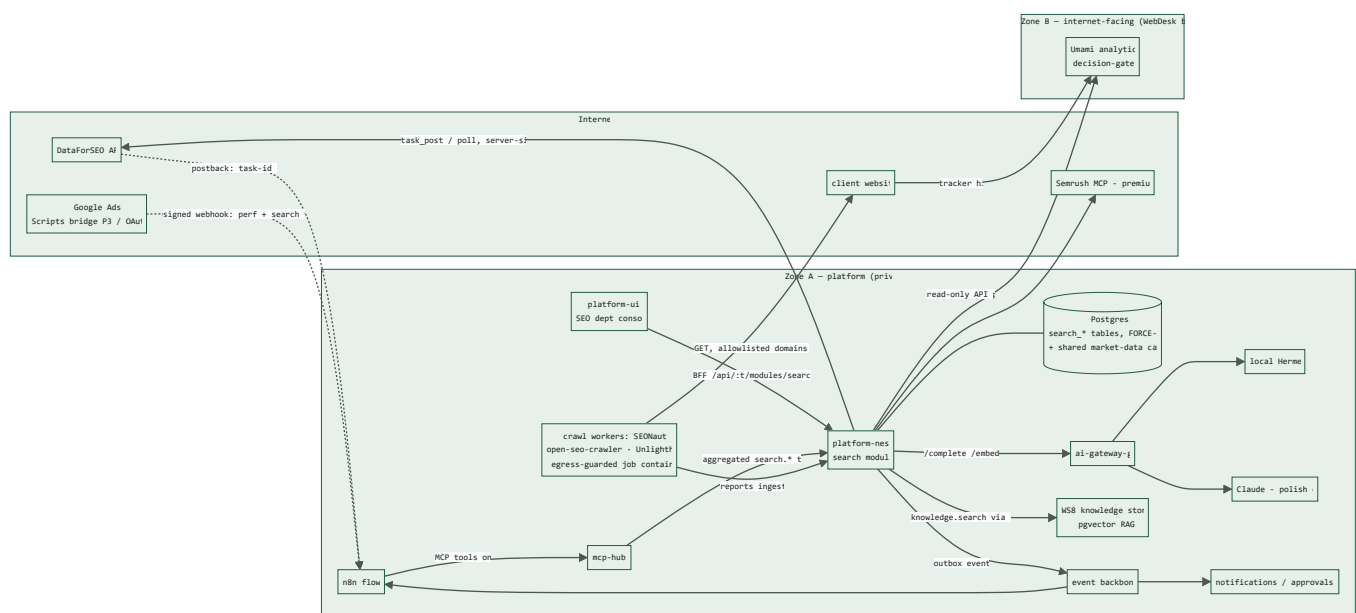


FIG. 03-1 — Master system diagram. Inbound webhooks are notification-only (task ids); the authoritative result is always fetched over the authenticated API. Umami sits in Zone B like everything internet-facing; the ERP only ever pulls from it.

Reading the diagram

- Every arrow that touches money or a live site passes through **platform-nest** — there is no path where n8n, the UI, or an agent calls a provider or a client site directly.

- **Crawler egress** is allowlisted per registered+verified property domain; **provider calls** use server-side keys only; **inbound webhooks** (DataForSEO postbacks, Ads-Scripts) are triggers only — the authoritative fetch always happens over the authenticated API.
- **Umami**, which must receive public traffic, follows WebDesk doctrine and sits in Zone B; the ERP only ever pulls read-only summaries.

§04 Trust Zones & Network

The WebDesk zone doctrine, adapted to this subsystem's actual exposure — almost everything lives in Zone A.

SURFACE	ZONE	RULES
<code>search</code> module + provider layer + ledger	A	Paid API keys (DataForSEO, Semrush) are server-side env → OpenBao target-state, held only by platform-nest. Never in the browser, never per-request from the UI, never in n8n credentials.
Crawl workers (SEONaut / open-seo-crawler / Unlighthouse)	A, egress-only	No inbound ports. May fetch only domains registered+verified on a <code>search_properties</code> row; deny RFC-1918/link-local/metadata IPs after DNS resolution (SSRF); respect robots.txt; identified UA; per-host rate cap.
DataForSEO postback / Ads-Scripts webhook	edge → n8n	HMAC-signed or secret-path, schema-validated, treated as triggers only . A forged webhook can cause at most a redundant authenticated fetch.
Umami (if approved)	B	Deployed with the WebDesk boxes; client sites embed its tracker. ERP holds a read-only API token and pulls; Umami has zero credentials into Zone A.
Client Google accounts (GSC/GA4/Ads — P4)	A vault	Per-client OAuth tokens ride the existing <code>integration_connections</code> vault (AES-256-GCM at rest, <code>hasToken</code> reads only) with <code>owner_kind</code> widened to <code>'client'</code> .

§05 Domain Model & Schema

Every tenant table carries the third-wall RLS predicate; the engagement is the outcome-tracked unit; the change-proposal is the dual-mode execution artifact.

Design rules (inherited, not optional)

- Every tenant table: `tenant_id uuid NOT NULL REFERENCES companies(id), origin_site, created_at/updated_at`, soft-delete `deleted_at` where user-facing.
- **FORCE-RLS with the WSD-3 third wall** — policy predicate `tenant_id = ANY(app_current_tenants()) AND app_module_allowed('search')` on both USING and WITH CHECK, byte-identical to `0028_module_hr.sql`.

- Migrations: next unused number (0034+ as of 2026-07-23); no in-migration GRANTS; no `sync_app` grants (search tables do not sync in v1).
- Money: minor units `bigint` + `currency` for client-facing money (SEM budgets/spend); **provider cost is `numeric(12,6)` USD** (DataForSEO unit prices go to \$0.00012).

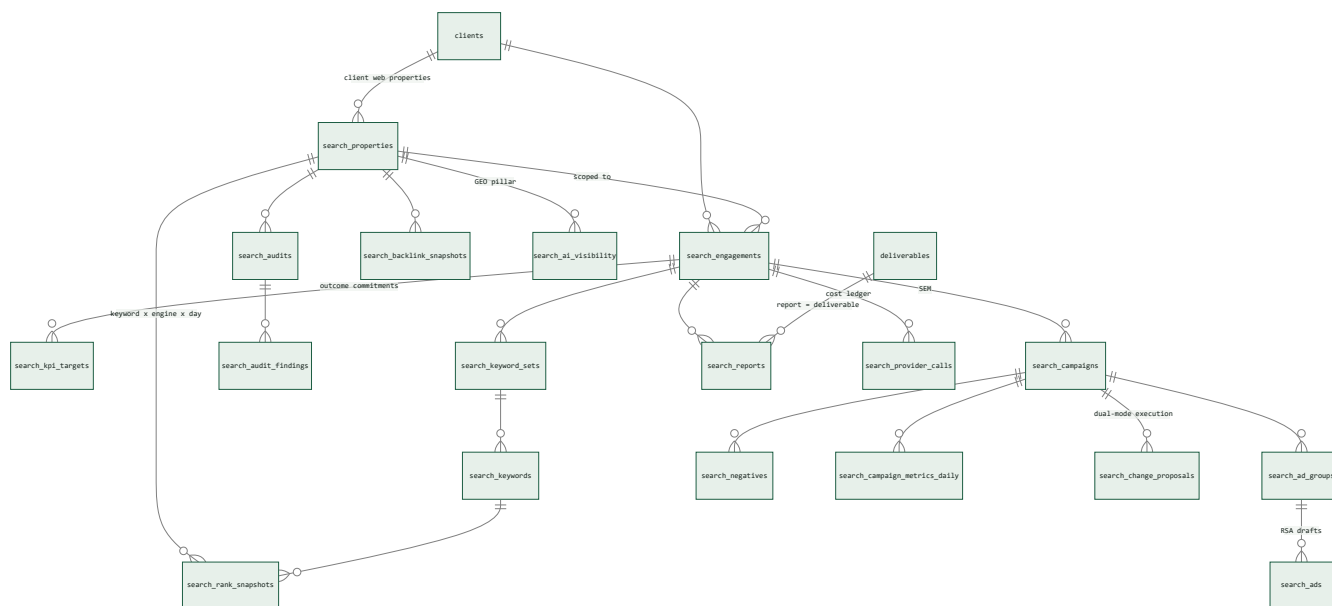


FIG. 05-1 – Entity map. `search_engagements` is the outcome-tracked unit; `search_change_proposals` is the single execution artifact every live-account action materializes as, before either twin applies it.

Key tables

TABLE	PURPOSE
<code>search_properties</code>	The client web property; anchor for crawls, ranks, GEO, analytics. Carries <code>domain</code> (crawl-allowlist source), <code>targets_jsonb</code> (engine/device/locale/location), optional GSC/GA4/Ads connection FKs (P4), <code>verified_at</code> (crawl-consent checkpoint).
<code>search_engagements</code>	The outcome-tracked engagement. Carries <code>scope_preset</code> (label only) and the real per-client <code>tool_scope_jsonb</code> — the human-set enable/cadence/limit config every scheduled flow and paid dispatch consults — plus <code>provider_budget_usd</code> stop-loss.
<code>search_kpi_targets</code>	Committed outcomes per engagement: <code>metric_key</code> (organic_sessions, top10_keywords, conversions, cpa_minor, roas_ratio, ai_citations...), baseline/target/due-period/direction. Reports measure against these.
<code>search_keyword_sets</code> / <code>search_keywords</code>	Keyword research + clustering. <code>embedding_vector(768)</code> is a dual-mode operational feature column (pgvector when present, <code>float8[]</code> fallback) — an operational feature, not a retrieval store.

TABLE	PURPOSE
search_rank_snapshots	Append-only time series: keyword × engine × device × location × day, <code>serp_features</code> <code>jsonb</code> (the GEO signal — <code>ai_overview</code> , <code>featured_snippet</code> , <code>local_pack</code> — rides here too).
search_audits / search_audit_findings	Crawl/CWV/content/links/geo audit kinds; findings carry severity, category, status, and regression tracking (diff of consecutive completed audits).
search_backlink_snapshots	Monthly aggregates only (totals, new/lost top-N samples) — full link inventory is deliberately not stored in v1.
search_ai_visibility	The GEO pillar: per engine (chatgpt/google_ai_overview/gemini/claude/perplexity), query, <code>brand_mentioned</code> , <code>cited</code> , <code>cited_url</code> , prominence.
search_campaigns / ad_groups / ads / negatives / metrics_daily	SEM planning objects: campaigns built from keyword clusters, RSA drafts, AI-proposed negatives, imported daily performance.
search_reports	Metrics snapshot + KPI-vs-target + AI narrative draft → human-edited → delivered as a <code>files</code> row mirrored to Shared Drive and linked into <code>deliverables</code> .
search_provider_calls	The metering ledger (\$06). <code>search_data_cache</code> is the shared market-data cache (\$06, deliberately no-RLS — D-4).

The dual-mode execution artifact

`search_change_proposals` is the mechanism that makes every live-account action safe regardless of which twin executes it — a launch, pause, budget/bid change, negatives batch, or ads batch is always a row first:

```
CREATE TABLE search_change_proposals (
  id uuid PRIMARY KEY,
  tenant_id uuid NOT NULL REFERENCES companies(id),
  campaign_id uuid NOT NULL REFERENCES search_campaigns(id),
  kind text NOT NULL CHECK (kind IN
    ('launch', 'pause', 'budget', 'bid', 'negatives_batch', 'ads_batch')),
  payload jsonb NOT NULL,          -- the exact intended change (hashed for approval ma
  status text NOT NULL DEFAULT 'proposed'
  CHECK (status IN ('proposed', 'approved', 'applied', 'dismissed')),
  mode text NOT NULL DEFAULT 'manual' CHECK (mode IN ('manual', 'api')),
  approval_id uuid REFERENCES automation_approvals(id), -- WS4 link (api mode; one-shot)
  export_file_id uuid REFERENCES files(id),    -- Ads-Editor-ready artifact (manual mode)
  proposed_by uuid REFERENCES users(id), approved_by uuid REFERENCES users(id),
  applied_by uuid REFERENCES users(id), applied_at timestampz,
  origin_site text NOT NULL, created_at timestampz NOT NULL DEFAULT now(),
  updated_at timestampz NOT NULL DEFAULT now(), deleted_at timestampz
);
```

Manual mode records who applied it in the ad platform (`applied_by/applied_at` , artifact attached); api mode executes only by consuming the one-shot approval (§08, §12). The human picks the mode per action — no approval, no API execution, including for humans in the console.

Custom fields & agency-vertical relations

- D17 `customFieldTargets` : `search_engagement` , `search_campaign` .
- **Clients** are the existing core `clients` rows — no duplicate client table.
- **Deliverables/time**: reports link into core `deliverables` ; hours log through the normal `time_entries` path against the optional `project_id` .

§06 Data-Provider Abstraction

The provider layer lives inside platform-nest — metering must be transactional with the ledger, RLS scoping is native, and no new deployable is created.

DataForSEO's async-queue model (Standard queue) posts a task and resolves by poll or postback-triggered fetch; live/sync providers resolve immediately. The interface is capability-based so drivers can be partial:

```
interface SearchDataProvider {
  key: 'dataforseo' | 'semrush' | 'scraper';
  capabilities: Set<'serp' | 'volume' | 'suggestions' | 'difficulty'
    | 'backlinks' | 'competitors' | 'ai_visibility'>;
  postSerpTasks(reqs: SerpRequest[]): Promise<TaskRef[]>;
  fetchSerpResults(refs: TaskRef[]): Promise<SerpResult[]>;
  getKeywordMetrics(kws: KeywordQuery[]): Promise<KeywordMetrics[]>;
  getBacklinkSummary(target: string): Promise<BacklinkSummary>;
  getAiVisibility(q: AiVisibilityQuery): Promise<AiVisibilityResult[]>;
  estimateCostUsd(op: ProviderOp): number;    // consulted BEFORE dispatch (stop-loss)
}
```

Selection order per call: engagement `tool_scope.provider` override → tenant default → platform default (`dataforseo`, Standard queue). Semrush serves premium engagements; the `scraper` driver (free autocomplete/PAA) is a capability-limited fallback for suggestions only.

Caching-in-Postgres

`search_data_cache` is **shared, no-RLS** (like `companies`): market data is public-world data, and the whole cost win is cross-tenant reuse — the same keyword volume serves N clients. Only the provider layer reads/writes it; it never serializes raw to tenant APIs; single-flight dispatch (advisory lock per cache key) prevents stampedes.

KIND	TTL
volume	30 days
serp	24 hours (tracked ranks bypass cache)
suggestions	14 days
backlinks	7 days
ai_visibility	7 days

D-4 · RATIFIED 2026-07-23

This is a deliberate, documented RLS exemption. It contains no client identifiers or client-private data — results are copied onto tenant rows, never returned raw.

The usage/cost ledger

Scope + budget stop-loss, checked in one place before dispatch: (0) the capability must be enabled in the engagement's `tool_scope` — a disabled tool is refused with the toggle named, regardless of budget; then month-to-date `SUM(cost_usd)` vs. engagement budget → tenant monthly cap → global platform cap (env, default \$150/mo until the deposit model is proven). Any breach: the call is refused, a `search.provider.budget_threshold` event fires (also at 80%), and the console shows the blocked state.

The ledger powers the per-engagement usage panel, tenant admin rollup, the `search.provider_cost.month` rollup metric (feeds billing), and the \$01 "blended \$/client" truth — **spend = Σ each engagement's enabled tools**. The scope panel (SM-29) shows a projected monthly cost per toggle so the human sees the price of each switch before flipping it.

§07 Fork/Adapt Plan per Repo

The monolith-adapt vs. data-only call, made per tool. "Data-only" means we take its data/engine/reference — never run its UI or auth.

REPO	VERDICT	KEEP	ADD / HOW IT PLUGS IN
every-app/open-seo	Data-only	DataForSEO client code + endpoint map; MCP tool shapes as a checklist; audit heuristics	Its DataForSEO wiring informs the <code>dataforseo</code> driver; tool list informs <code>search.*</code> MCP tools
SEONaut (Go+MySQL)	Adapt, job-mode	Crawl engine, severity-ranked audit rules	Job container (+ ephemeral MySQL sidecar); adapter parses report → <code>search_audit_findings</code> . Fallback spike: port rules into the ingest adapter if the sidecar proves too heavy
open-seo-crawler	Adapt, job-mode	Concurrent crawler, sitemap hygiene, exports	Job container; XLSX/JSON export → ingest adapter; crawled content also feeds WS8 knowledge ingest
Unlighthouse	Data-only CLI, job-mode	Site-wide Lighthouse/CWV JSON	n8n-scheduled CLI job → JSON → <code>search_audits(kind='cwv')</code>
SerpBear	Data-only (reference)	Its pluggable SERP-scraper adapter list	Informs the <code>scraper</code> fallback driver; rank UX informs the Rankings tab
Umami	Run stock as shared infra	Everything (stock container)	One Umami website per <code>search_properties</code> ; ERP stores read-only API token. Decision-gated (OQ-5)

REPO	VERDICT	KEEP	ADD / HOW IT PLUGS IN
Claude Ads	Integrate as MCP tooling, phased	Audit/budget-check/draft-first patterns, scripts	P3: Scripts patterns → read bridge + manual-apply twin. P4: MCP/API surface executes api-mode proposals behind WS4

§08 AI Design

Local-first task routing, the AI-drafts→human-approves→execute spine, and search data exposed as MCP tools.

TASK	MODEL	TRIGGER
Keyword/intent clustering	gateway <code>/embed</code> (local) + Hermes labels	Keywords tab, bulk
Search-term → negative classification	Hermes + rules	Weekly sweep / import
Audit-finding triage & fix drafts	Hermes	Post-audit
Content briefs (+ GEO extractability)	Hermes draft → Claude optional polish	Briefs tab, RAG via WS8
Report narrative	Hermes draft → Claude polish where scoped	Monthly flow — the one place cloud polish defaults on
RSA ad copy	Hermes draft → Claude final for approved launches	Ads Studio — drafts never auto-publish

PGVECTOR NOTE (D9 HYGIENE)

`search_keywords.embedding` is an operational feature column for clustering, not a retrieval store — it copies the WS8 dual-mode pattern (pgvector when the extension exists, `float8[]` fallback otherwise). Anything retrieval-shaped (briefs, GEO corpus) goes through WS8.

AI-drafts → human-approves → execute

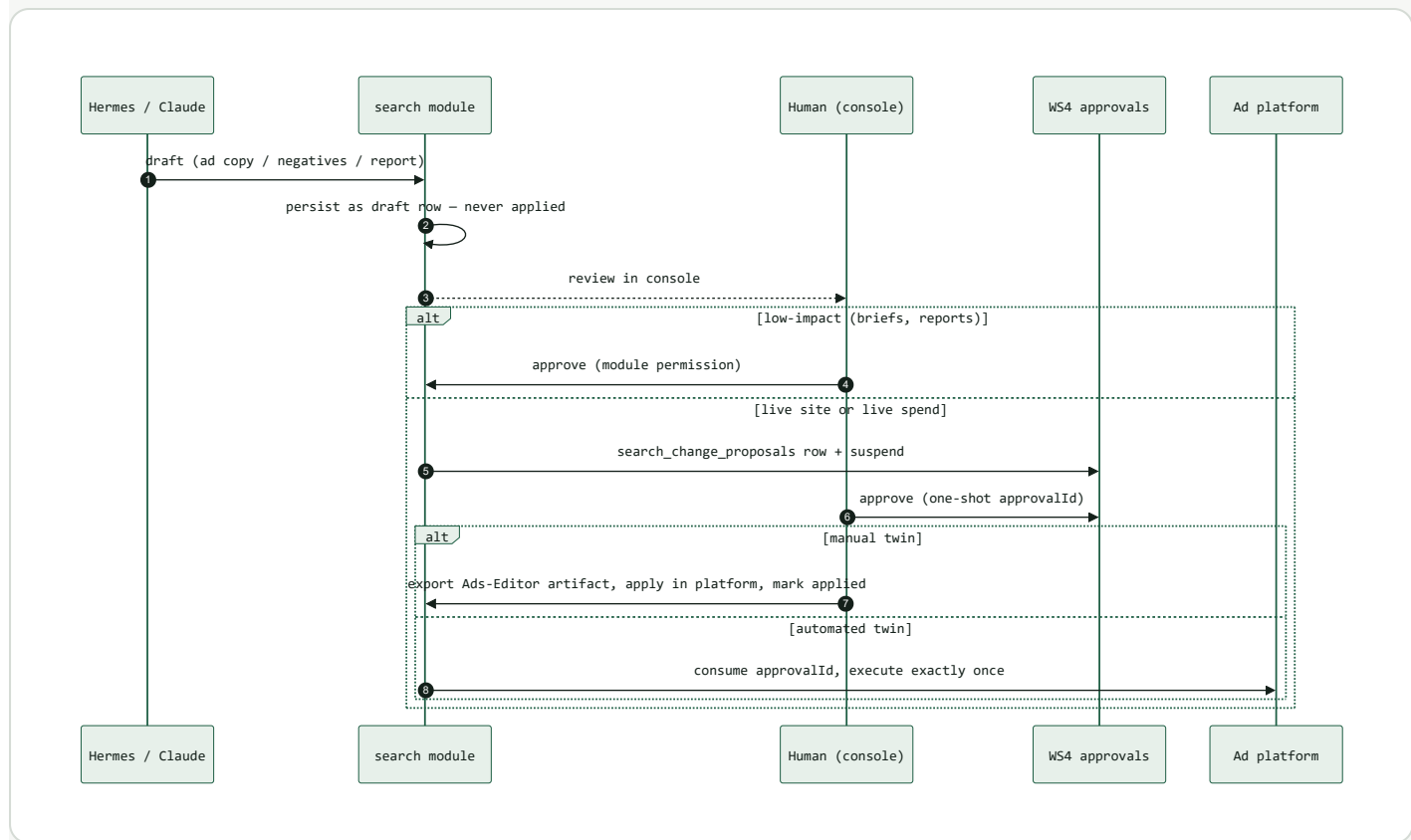


FIG. 08-1 – No approval, no execution – including for humans in the console. The one-shot approvalId avoids the deferred Temporal-resume problem the same way WS4 did.

Search data as MCP tools

Registered via `ModuleContract.mcpTools` (aggregated by mcp-hub — nothing hub-side to hardcode). Reads are `minAssurance: 'low'`; paid-data pulls are declared `write: true, impact: 'medium'` even though they're semantically reads — they spend money — so automation principals route through the D14 gate and the ledger records the OBO automation user.

TOOL	KIND	IMPACT
search.listEngagements / rankSummary / auditSummary / ledgerSummary	read	—
search.keywordResearch	paid pull	medium (budget-checked)
search.pullRanks / pullBacklinks / pullAiVisibility	paid pull	medium
search.runAudit	job trigger (\$0)	low
search.clusterKeywords / draftBrief / proposeNegatives / draftReport	AI draft	low (drafts only)
search.exportProposal	export (manual mode)	low (no live side effect)

TOOL	KIND	IMPACT
search.applyNegatives / setBudget / launchCampaign / publishContent	live mutation (api mode)	high → always suspends to WS4

§09 Console UX

One department console, dept name **"SEO"**, on the dept-interface-template exactly as Web Dev uses it — universal spine plus three craft groups.

GROUP	SUB-TABS (UNDER /DEPARTMENTS/SEO/)
Accounts	Engagements (engagements) · Reports (reports)
Optimize (SEO+GEO)	Site Audit (audit) · Keywords (keywords) · Rankings (rankings) · Content Briefs (briefs) · AI Visibility (ai-visibility)
Campaigns (SEM)	Planner (planner) · Ads Studio (ads) · Search Terms (search-terms) · Pacing (pacing)

Home is the command-center template: a KPI strip from rollup metrics (active engagements, top-10 keywords, critical findings, MTD provider spend, MTD ad spend), an activity feed, and a launcher row (GSC, GA4, Google Ads, Looker Studio, Claude). The My-Work rail is inherited unchanged; Connections gains the property/provider link states.

Button capability matrix

Every  action is additionally gated by the engagement's **tool-scope config**: if the human hasn't enabled that tool for the client, the button renders disabled and names the missing toggle. Every live-account action is **dual-mode** — the manual twin and the automated twin appear side by side; the human picks per action.

-  AI/crawler-only – \$0 external, buildable now
-  DATA KEY – DataForSEO/Semrush, ledger/budget-checked
-  ADS LINK – client ad-account OAuth (P4)
-  WS4 APPROVAL – human decision required

CONSOLE ACTION	TAB	NEEDS	GATE
Create engagement / property / KPI targets	Engagements	—	permission only
Configure engagement scope & budget (toggle grid + cadence + caps + projected cost)	Engagements	—	search:scope:write
Run technical audit (SEONaut/crawler)	Site Audit		low

CONSOLE ACTION	TAB	NEEDS	GATE
Run CWV scan (Unlighthouse)	Site Audit		low
AI triage findings / draft fixes	Site Audit		draft only
Import keywords (CSV / paste / GSC export)	Keywords		—
AI cluster + intent-tag set	Keywords		draft only
Fetch volume / difficulty / CPC	Keywords		budget stop-loss
Keyword suggestions (autocomplete/PAA)	Keywords	(scraper) / richer	budget if
Track keywords / Pull ranks now	Rankings		budget stop-loss
Backlink snapshot	Rankings		budget stop-loss
Draft content brief (RAG over own crawl)	Briefs		draft only
Polish brief/content with Claude	Briefs	 (gateway-capped)	draft only
Publish content to client site (WebDesk)	Briefs	WebDesk P5 seam	
Pull AI visibility (GEO)	AI Visibility		budget stop-loss
Build campaign plan from clusters	Planner		draft only
Generate RSA drafts	Ads Studio		draft only
Import performance CSV	Pacing		—
Live perf/search-term sync (Scripts bridge P3 / OAuth P4)	Pacing / Search Terms	bridge or	— (read)
AI propose negatives	Search Terms		draft only
Apply negatives — manual twin : export + mark applied	Search Terms		search:campaign:launch
Apply negatives — automated twin : API push	Search Terms		
Budget / bid / pause — manual twin	Pacing		search:campaign:launch

CONSOLE ACTION	TAB	NEEDS	GATE
Budget / bid / pause — automated twin	Pacing		
Launch campaign — manual twin : export build sheet	Planner		search:campaign:launch
Launch campaign — automated twin	Planner		
Generate report (metrics + AI narrative)	Reports	(enriches)	search:report:approve
Deliver report (Shared Drive + deliverable)	Reports		after approve
View usage ledger / raise engagement budget	Engagements	—	search:ledger:read / search:provider:admin

Everything ships value in P1 with zero external spend — that is deliberate.

§10 ERP Integration Points

Reuse over invention — no new subsystem is created; this vertical plugs into every existing plane the platform already has.

SUBSYSTEM	INTEGRATION
platform-nest	<code>ModuleContract</code> key <code>search</code> ; hr-convention controller path; <code>ModuleEnabledGuard</code> ; enablement via <code>enabled_modules</code> OR active <code>service_assignment</code>
BFF contract	New section in <code>FRONTEND-BFF-CONTRACT.md</code> : <code>/api/:t/modules/search/{properties,engagements,.../scope,kpi-targets,keyword-sets,keywords,ranks,audits,findings,backlinks,ai-visibility,campaigns,ad-groups,ads,negatives,change-proposals,metrics-daily,reports,ledger,research/*}</code>
mcp-hub	<code>search.*</code> tools via <code>mcpTools</code> aggregation (\$08); no hub changes needed
ai-gateway-go	<code>/complete</code> (Hermes default, Claude flagged) + <code>/embed</code> ; gateway budget caps + DLP + egress audit apply automatically
WS8 knowledge	Crawled site content ingested as tenant-ACL'd sources; briefs/GEO retrieval via <code>knowledge.search</code> (D9 preserved)
automation (n8n)	Flows in §11; n8n orchestrates, MCP accesses, zero logic in workflows
WS4 approvals	High-impact <code>search.*</code> tools suspend into <code>automation_approvals</code> ; api-mode proposals execute only with a one-shot approved <code>approvalId</code>

SUBSYSTEM	INTEGRATION
Event backbone	<code>search.rank.dropped</code> , <code>audit.completed/regression</code> , <code>backlinks.lost_spike</code> , <code>budget.overspend</code> , <code>provider.budget_threshold</code> , <code>report.ready_for_review/delivered</code> , <code>campaign.proposed</code> , <code>ai_visibility.changed</code> → notifications bell + n8n bridge
Files / Shared Drive (WS11)	Raw crawl exports + rendered reports as <code>files</code> rows mirrored to the client's Drive folder
Rollups (D12)	<code>search.engagements.active</code> · <code>search.rank.top10</code> · <code>search.audits.critical_open</code> · <code>search.provider_cost.month</code> (money, isMonetary) · <code>search.sem_spend.month</code> · <code>search.reports.delivered</code>
Cerbos	New resource policies (§12); UI capabilities mirrored in <code>lib/rbac.ts</code>
observability (WS9)	OTel spans on provider calls (provider, endpoint, items, cost_usd, cache_hit), crawl jobs, AI calls; ledger-vs-cap gauge
integration_connections (0033)	Widen <code>provider</code> CHECK + <code>owner_kind 'client'</code> (polymorphic owner_id → clients.id). Tokens only via <code>setConnectionTokens</code> (P4 OAuth)
WebDesk (Zone B)	<code>search.publishContent</code> targets the WebDesk content engine for WebDesk-hosted clients (post-WebDesk-P3); until then briefs export as files

§11 Automation Flows

All flows are thin orchestrations calling `search.*` MCP tools. Scheduling is scope-driven — each flow iterates only engagements whose `tool_scope` enables the tool at the due cadence.

FLOW	SCHEDULE / TRIGGER	MCP CALLS	PHASE
<code>sm-site-audit</code>	weekly per active property	<code>search.runAudit(kind='technical')</code> → <code>auditSummary</code> → notify	P1
<code>sm-cwv-scan</code>	weekly	<code>search.runAudit(kind='cwv')</code>	P1
<code>sm-rank-pull</code>	cadence from <code>tool_scope</code>	<code>search.pullRanks</code> (Standard tasks; scope+ledger-checked)	P2
<code>sm-rank-collect</code>	webhook (task-id only) + safety poll	<code>search.ingestRankResults</code> (authoritative API fetch)	P2
<code>sm-keyword-refresh</code>	monthly	<code>search.keywordResearch</code> (cache-aware)	P2
<code>sm-backlink-snapshot</code>	monthly	<code>search.pullBacklinks</code>	P2
<code>sm-ai-visibility</code>	weekly, per scope	<code>search.pullAiVisibility</code>	P2

FLOW	SCHEDULE / TRIGGER	MCP CALLS	PHASE
sm-provider-ledger-guard	daily	<code>search.ledgerSummary</code> → threshold events → notify/pause	P2
sm-search-term-sweep	weekly	ingest terms → <code>proposeNegatives</code> → change proposals	P3
sm-budget-pacing	daily	pacing calc → <code>budget.overspend</code> ; proposed pause = a change proposal	P3
sm-monthly-report	monthly per engagement	<code>draftReport</code> → review → on approve <code>deliverReport</code>	P3

§12 Trust & Security

Nothing new to invent — every control reuses an existing platform mechanism, applied to a new set of resources.

KEY CUSTODY

DataForSEO/Semrush keys: platform-nest env → OpenBao target-state; never in platform-ui, n8n credential stores, or per-tenant rows. Client OAuth tokens (P4) ride the `integration_connections` vault only.

RLS

All `search_*` tenant tables: FORCE-RLS, third-wall predicate, fail-closed empty-set semantics. The single exemption is `search_data_cache` — ratified, D-4.

CERBOS RESOURCES

New policy files: `resource_search_property/engagement/keyword/audit/campaign/report/ledger`. `resource_search_engagement` carries a `set_scope` action — the tool-scope decision is itself permission-gated.

CRAWLER EGRESS (SSRF)

Resolve-then-check against registered+verified property domains; deny private/link-local/metadata ranges; robots.txt respect; rate caps; identified UA. Property `verified_at` is a prerequisite to first crawl.

INBOUND WEBHOOKS

Notification-only (task ids), HMAC/secret-path validated at n8n; authoritative data always re-fetched over the authenticated provider API.

MONEY SAFETY

Single dispatch choke-point (estimate → ledger check → advisory-lock → dispatch → true-up); one-shot approval; consumption is replay-proof.

Module permissions declared in the contract: `search:engagement:read|write` , `search:scope:write` , `search:keyword:write` , `search:rank:read` , `search:audit:run` , `search:brief:write` , `search:campaign:write` ,

`search:campaign:launch` (covers manual mark-applied *and* api execution), `search:content:publish`,
`search:report:write|approve`, `search:ledger:read`, `search:provider:admin`.

Audit: every provider call (ledger), every Cerbos decision (existing), every hub tool call (existing JSONL), every approval decision (existing) — nothing new to invent.

§13 Rollout & Ticket Decomposition

Cost-honest phases — P0 contracts → P1 zero-spend value → P2 paid data → P3 SEM + reports → P4 committed live-ads automation. 26 tickets P0–P3, 2 committed P4, /army-ready.

P0

Foundation

Migrations for all \$05 tables + third-wall RLS + `search_data_cache` (no-RLS) · ModuleContract + CRUD + tool-scope endpoints · Cerbos policies × 7 · provider abstraction + ledger + stop-loss choke-point · DataForSEO driver · config plumbing.

GATE → module registers, RLS proven, budget stop-loss refuses correctly before any real spend.

P1

\$0 value — crawlers + AI on own data

Crawl workers with egress guard · audit ingest + regression diff · keyword import/embed/cluster · AI drafting services · console shell + Optimize tabs v1 · engagement scope-config surface.

GATE → register a property → crawl → findings → triage → cluster, all in-console, zero external spend.

P2

Paid data (gate: OQ-2 deposit)

Rank tracking pull/ingest + Rankings UI · n8n flows batch 1 · backlink snapshots + AI-visibility pulls · metering surfaces (usage panel, admin summary, threshold events).

GATE → a real DataForSEO pull renders in Rankings; ledger sums reconcile with a scripted call sequence.

P3

SEM + reports

SEM domain + change-proposal model · manual-apply path (Ads-Editor exports) · Campaigns craft-group UI with the dual-mode picker · Ads-Scripts read bridge · WS4 execution path (one-shot approvalId) · reports · docs/registration · full-stack e2e.

GATE → the full planning loop runs in-console; both twins render; MODULES.md bumped to DEV-VERIFIED.

P4

Live-ads automation (committed) + decision-gated extras

Google OAuth (GSC/GA4/Ads) on the 0033 vault · automated live-write executor consuming the one-shot approvalId. Decision-gated: Umami deploy (OQ-5), Semrush premium driver (OQ-3).

GATE → a sandbox/test-account push applies exactly the proposal payload; ledger/audit rows complete.

P0 — Foundation

#	TICKET	TIER	MODEL	DEPS
SM-01 ⚡	Migration(s): all \$05 tables + third-wall RLS + indexes + <code>search_data_cache</code> (no-RLS, D-4) + dual-mode embedding + <code>integration_connections</code> widen	senior-db	opus·medium	—
SM-02 ⚡	<code>search</code> ModuleContract + controller skeleton + registry + permissions + guard + uiManifest + engagement/property/kpi CRUD + tool-scope endpoints + presets	senior-be	default	SM-01
SM-03 ⚡	Cerbos policies × 7 + derived-roles wiring + policy tests + <code>lib/rbac.ts</code> mirror	medior	default	SM-02
SM-04 ⚡	Provider abstraction: <code>SearchDataProvider</code> , capability registry, cache layer (TTLs, single-flight), scope+ledger+stop-loss choke-point, cost projection endpoint	senior-be	opus·medium	SM-01,02
SM-05	DataForSEO driver: Standard task_post/poll + postback-triggered fetch; SERP/volume/labs/backlinks/ai-visibility endpoints	senior-be	default	SM-04
SM-06	Config plumbing: server-side creds, caps env, per-pillar feature flags, <code>.env.example</code>	junior	default	SM-04

P1 — \$0 value

#	TICKET	TIER	MODEL	DEPS
SM-07	Crawl workers: SEONaut job-mode (+ MySQL sidecar spike w/ open-seo-crawler fallback), open-seo-crawler, Unlighthouse runner; egress guard	senior-integrator	default · QA gate	SM-01
SM-08	Audit ingest adapters, regression diff, events	medior	default	SM-07,02
SM-09	Keywords: import, embeddings, clustering (dual-mode), Hermes intent/labels	medior	default	SM-02
SM-10	AI drafting services: briefs (WS8 RAG), finding triage, meta suggestions, narrative draft	senior-be	default	SM-08,09
SM-11 ⚡	Console shell: <code>seo</code> toolkit (3 craft groups), tab skeletons, <code>lib/search.ts</code> BFF types	senior-fe	default	SM-02
SM-12	Optimize tabs v1: Site Audit, Keywords (import/cluster; scope-disabled states)	medior	default	SM-11,08,09
SM-13	Events → notifications wiring for all \$10 event types	junior	default	SM-08
SM-29	Engagement scope-config surface: toggle grid, preset picker, projected monthly cost, budget-cap editor	medior	default	SM-11,04

P2 — Paid data (gate: OQ-2 deposit)

#	TICKET	TIER	MODEL	DEPS
SM-14	Rank tracking: pull/ingest endpoints + Rankings UI	medior	default	SM-05,11
SM-15	n8n flows batch 1 (rank-pull/collect, site-audit, cwv-scan, keyword-refresh) + seeds	medior	default	SM-05,08
SM-16	Backlink snapshots + AI-visibility pulls + their panels	medior	default	SM-05,11
SM-17	Metering surfaces: engagement usage panel, tenant admin summary, threshold events, rollup metrics	medior	default	SM-04,11

P3 — SEM + reports

#	TICKET	TIER	MODEL	DEPS
SM-18	SEM domain: campaigns/ad-groups/ads/negatives CRUD, change-proposal model, cluster→plan generator, RSA + negative AI drafts	senior-be	default	SM-09
SM-30	Manual-apply path: approved proposal → Ads-Editor-ready exports → mark-applied flow	senior-be	default	SM-18
SM-19	Campaigns craft-group UI (Planner, Ads Studio, Search Terms, Pacing) with the dual-mode picker	senior-fe	default	SM-18,30,11
SM-20	Ads-Scripts read bridge: exporter template + signed n8n webhook → metrics-daily + search terms	senior-integrator	default	SM-18
SM-21 ⚡	WS4 execution path: high-impact tools registered, one-shot approvalId consumption, console execute-after-approve UX	senior-be	opus·high	SM-18,03
SM-22	Reports: snapshot + narrative draft → review/approve → render → files/Drive + deliverable link; monthly/pacing/sweep flows	medior	default	SM-10,17,18
SM-23	Docs/registration: MODULES.md flip + CHANGELOG, BFF-contract rows, runbook stub, dept-console IA plan update	junior	default	SM-02
SM-24	Full-stack e2e verification on the live dev stack + Playwright console suite	medior	default	all

P4 — Live-ads automation (committed) + decision-gated

#	TICKET	TIER	MODEL	DEPS
SM-25 ⚡	Google OAuth (GSC/GA4/Ads) on the 0033 vault: consent flows, token refresh, <code>owner_kind='client'</code> links	senior-be	opus·medium	SM-01
SM-26	Automated live-write executor: approved api-mode proposals → Google Ads API push, one-shot consumption, rollback notes	senior-integrator	default	SM-21,25

Decision-gated (do not mobilize): SM-27 Umami deploy + property binding [senior-integrator] (OQ-5) · SM-28 Semrush premium driver [medior] (OQ-3).

Count by tier (P0–P3, 26 tickets): senior-db 1 · senior-be 7 · senior-fe 2 · senior-integrator 2 · medior 11 · junior 3. **P4 committed adds:** senior-be 1 · senior-integrator 1. **Opus flags: 4** (SM-01 med, SM-04 med, SM-21 high, SM-25 med).
 Concurrency: respect the 1–2 agent cap; the only safe parallel pairs early are (SM-03 || SM-04) and (SM-07 || SM-09).

⚡ = touches a contract (schema/API/policy) → QA gate + architect design-review on the diff.

§14 Open Questions & Decision Log

Four owner decisions resolved this design (v1.1); four remain, none blocking P0/P1.

RESOLVED BY THE OWNER 2026-07-23 (V1.1)

ex-OQ-1 (console IA + name → D-10) · ex-OQ-4 (SEM dual-mode → D-8) · ex-OQ-6 (per-engagement tool scope → D-11) · ex-OQ-7 (shared cache ratified → D-4).

Remaining open questions

#	QUESTION	BLOCKS	DEFAULT IF UNANSWERED
OQ-2	Fund the \$50 DataForSEO deposit now? (P0/P1 proceed regardless)	P2	Build P0/P1, hold P2
OQ-3	Semrush MCP OAuth — authorize now or at first premium client? (Connector currently unauthorized)	SM-28	Defer to premium demand
OQ-5	Umami pillar: deploy (Zone B WebDesk boxes vs own VPS) or defer — GSC+GA4 may suffice?	SM-27	Defer
OQ-8	pgvector extension in the platform DB images (dual-mode fallback keeps plain-PG tests green)	SM-01/09	Add to dev/VPS images

Decision log

LOCKED UPSTREAM (FOUNDATION §01 – NOT RELITIGATED HERE)

DataForSEO Standard primary + Semrush premium behind a pluggable abstraction; local-Hermes-first AI via the gateway; self-hosted crawlers for \$0 audit/crawl/speed; Postgres SERP/volume caching; per-client metering on a shared deposit; ~\$8–10/client/mo blended · ~\$900/mo central at 100 clients.

#	DECISION	WHY
D-1	Provider layer lives inside platform-nest (no new service; open-seo not run)	Transactional metering, native RLS, no deployable sprawl
D-2	Per-repo verdicts as §07 (open-seo + SerpBear data-only; SEONaut/crawler/Unlighthouse job-mode; Umami stock infra; Claude Ads via MCP behind WS4)	Multi-tenant retrofit cost vs. thin data calls; no external UIs

#	DECISION	WHY
D-3	Module key <code>search</code> , tables <code>search_*</code> , hr-style controller path + third-wall RLS	Newest-vertical (hr) conventions
D-4	Shared no-RLS market-data cache, service-layer-only. Ratified 2026-07-23	The cross-tenant reuse is the cost model
D-5	Paid-data pulls declared <code>write:true, impact:'medium'</code> MCP tools	Spending money is a mutation; automation must hit the D14 gate
D-6	Api-mode executions require one-shot approved <code>approvalId</code> (humans included)	Closes the WS4 no-Temporal-resume gap without new infra
D-7	Keyword embeddings as a dual-mode operational column; retrieval-shaped RAG stays in WS8	D9 ownership preserved; clustering can't round-trip MCP
D-8	Dual-mode SEM execution: every live action is a <code>search_change_proposals</code> row; manual twin (P3, zero OAuth) and automated twin (P4 committed); human picks the mode	Best of both: early value with no OAuth on the critical path, full automation when OAuth lands
D-9	Umami in Zone B if approved; ERP pulls read-only	Internet-facing = WebDesk zone doctrine
D-10	Console ratified: dept name stays SEO; console = dept-interface-template exactly as Web Dev, 3 craft groups as primary-strip divisions	One console, WebDev-proven pattern
D-11	Per-engagement tool-scope config: every tool/paid pull enabled per client via <code>tool_scope</code> + per-engagement budget caps; fixed tiers survive only as presets	Fully custom per-client invocation; cost = Σ enabled tools per engagement

§15 Glossary

TERM	MEANING
GEO / AEO / LLMO	Generative/Answer-Engine Optimization — being cited by AI answer engines (ChatGPT, AI Overviews, Gemini, Claude, Perplexity)
Engagement	The outcome-tracked unit of client work — KPI targets, scope, budget — not a task list
Tool scope	The per-engagement, human-set config of which tools/paid pulls run for a given client, at what cadence and limit
Change proposal	The single dual-mode execution artifact every live-account action materializes as, before either twin applies it

TERM	MEANING
Manual twin	Approved proposal exports Ads-Editor-ready; a human applies it in the ad platform — zero OAuth
Automated twin	ERP pushes the approved proposal via API behind WS4, consuming a one-shot approval id
Data-only	We take a tool's data/engine/reference as a build input; we never run its UI or auth
Third-wall RLS	FORCE-RLS predicate combining tenant scope with per-module enablement, per the hr-migration convention
Stop-loss	The pre-dispatch check (scope → engagement budget → tenant cap → global cap) that refuses a paid call before money is spent
DataForSEO Standard queue	The async, ~5-minute, 3×-cheaper-than-Live SERP task queue used as the default provider mode

GAIADA Search-Marketing — Engineering Blueprint v1.1

Status: Design blueprint · Planned (v0.0.0) · 2026-07-23 · SEO Dept · Platform · Classification: Internal

Architecture conforms to the locked foundation cost model (§01). Feeds /army decomposition (§13). Not a commitment of dates. Cross-references: [docs/blueprints/seo-sem-design.md](#) · [docs/blueprints/seo-sem-foundation.md](#) · [docs/BLEUPRINTS.md](#) .